PROJECT PORTFOLIO / 2026

牟洪毅

AI Agent 与 Python 后端工程作品集

东北大学控制工程硕士在读 | 青岛科技大学机器人工程本科
方向：Agent 系统、RAG / GraphRAG、异步服务与交易一致性

GitHub 主页

工程主线

从 LLM 工具编排与知识检索，到多阶段研究工作流，再到高并发订单与异步支付：三个项目覆盖 AI 应用层和可靠后端链路。每个项目页都提供仓库、关键代码入口和可核验的运行记录。

三项项目速览

01

Enterprise GraphRAG Agent

LangGraph · Hybrid RAG · Neo4j

7 个 Demo 案例 | 整体通过率 100%*

打开仓库 ↗

02

Deep Research Agent

Planner · Parallel Research · Evidence

DDGS 搜索 | 引用校验 | Checkpoint / HITL

打开仓库 ↗

03

RailFlow

FastAPI · Bitmap · MySQL / Redis / RabbitMQ

20 并发：1 成功、19 冲突 | 40 项测试通过*

打开仓库 ↗

* GraphRAG 数字来自 7 条 Demo Evaluation；RailFlow 为本地项目验证记录，自动化测试主要覆盖 SQLite / 内存环境，不等同于生产压测。Deep Research 仓库定位为学习与演示原型，未展示可公开核验的总体效果分数。

PROJECT 01

Enterprise GraphRAG Agent

面向企业知识库问答的工程化 Agent：让检索只在需要时发生，并为答案组织文本与图关系证据。

核心链路

FastAPI – LangGraph ReAct – Tool Routing – Hybrid RAG – Neo4j Graph – Context + Memory

解决的问题与实现

• Agent Runtime：LangGraph ReAct 状态图支持 Agent → Tool → Observation → Agent 闭环；仅暴露 `search_knowledge`，将检索策略封装在工具内部。
• 混合证据：Vector + BM25 双路召回，经 RRF 排名融合和 Cross-Encoder 精排；并行接入 Neo4j 实体、关系与路径证据。
• 上下文与韧性：按 thread 管理短期记忆、按 user 管理长期信息；Timeout / Retry / Backoff / Semaphore 控制依赖波动，Neo4j 异常时保留文本检索链路。

100%	7	1,920.89 ms
Overall Pass Rate	Demo Evaluation Cases	Average Latency

口径说明：以上是仓库当前 7 个 Demo Case 的一次评测记录，用来验证评测链路；样本规模较小，不代表生产 Benchmark。仓库也明确标注 Vector Store、Memory 与 Graph 抽取仍是原型级实现。

招聘人员可直接查看

LangGraph 状态图 ↗
知识库工具 ↗
检索模块目录 ↗
评测入口 ↗
Demo Snapshot ↗

打开完整仓库 ↗

PROJECT 02

Deep Research Agent

面向复杂问题的多阶段研究与报告生成原型：把一次长回答拆成可检查、可暂停、可恢复的工作流。

执行闭环



可靠性与人工控制：Timeout / Retry / Semaphore · JSON Checkpoint · Human-in-the-loop

系统能力

- 多阶段协作：Planner 将复杂问题拆分为研究任务；Researcher 通过 DDGS Function Tool 联网搜索，并以 asyncio 并发执行。
- 证据约束：Pydantic 结构化任务、来源目录与 Verifier 检查；Citation Guard 约束最终报告中的引用编号与 URL。
- 可恢复运行：对研究调用做超时、重试和并发限制；JSON Checkpoint 支持暂停恢复，报告交付前可走人工审核。
- 工程入口：仓库包含 workflow、tools、verification、checkpoint、reliability 和离线评测模块，另有规则测试脚本。

结果与边界

仓库状态为“学习与演示原型”。README 提供 `python run.py` 与 `python evaluate_run.py` 入口，但公开仓库没有展示可核验的总体效果分数；离线指标是任务/引用规则代理指标，不等同于事实正确率。

代码与运行入口

工作流编排 ↗

搜索工具 ↗

来源验证 ↗

测试脚本 / 评测入口 ↗

README 与运行说明 ↗

打开完整仓库 ↗

PROJECT 03

RailFlow

区间库存与异步支付票务系统：用位图表达可复用座位区间，并把并发正确性落到数据库事务与订单状态。

订单与库存主链路

FastAPI › Order / Seat Service › Redis Lock + MySQL Row Lock › Bitmap Inventory › RabbitMQ Payment

关键设计

• Bitmap 区间库存：每个 bit 表示一个相邻站段；按位与检测区间冲突，按位或占座，支持同一座位的非重叠区间复用。
• 并发与幂等：Redis 锁降低热点竞争，MySQL `SELECT FOR UPDATE` 行锁与事务兜底；`(user_id, request_id)` 联合唯一约束避免重复订单。
• 异步支付与关单：HMAC-SHA256 模拟回调经 RabbitMQ 消费；状态机处理重复消息，过期订单释放前检查位图缺位与重叠风险。

运行结果记录

1 / 20
并发不同请求成功；其余 19 个冲突

1 笔订单
20 次相同幂等请求复用同一订单

40 passed
本地自动化测试记录

测试口径：40 项自动化测试以 SQLite / 内存测试为主；MySQL、Redis、RabbitMQ 的端到端联调另行进行。项目当前是模拟支付，不代表真实资金结算或生产级压力测试。

代码入口与可复现命令

```
# seat_bitmap.py

return (seat_bitmap & request_mask) == 0
```

位图实现 ↗

订单服务 ↗

并发实验脚本 ↗

测试目录 ↗

```
■■■python -m pytest -q
```

打开完整仓库 ↗